pe_host_link Implementation Specification

Nov 7, 2026 · @Aries X

1. Status and scope

pe_host_link is frozen: an SPI target (mode 0) that turns one 40-bit frame into one 16-bit access on the system bus as master 0. It uses 48 flops (about 3.8k μm²) in the single `clk` domain. This document is the implementation spec for RTL; it adds exact equations, state machines, timing, area and the verification plan to the micro-architecture decisions.

The equations were run through a Python gate-equation model on the same benches as the cycle model, and all pass (§8). Nothing here changes behaviour visible to programs or pins beyond the two approved changes below.

ID	Approved change (owner, 7 Nov 2026)	Replaces	Reason
A-CHG-1	Address is sent before the command byte	HL-03 order: command, address, data	The read bit arrives with the address complete, giving 7 SCK of turnaround and no dummy word
A-CHG-2	One data word per frame, no bursts, no auto-increment	Arch §3 auto-increment, HL-03 burst, VER-07 burst	No incrementer, no prefetch, so FIFO B is never read speculatively

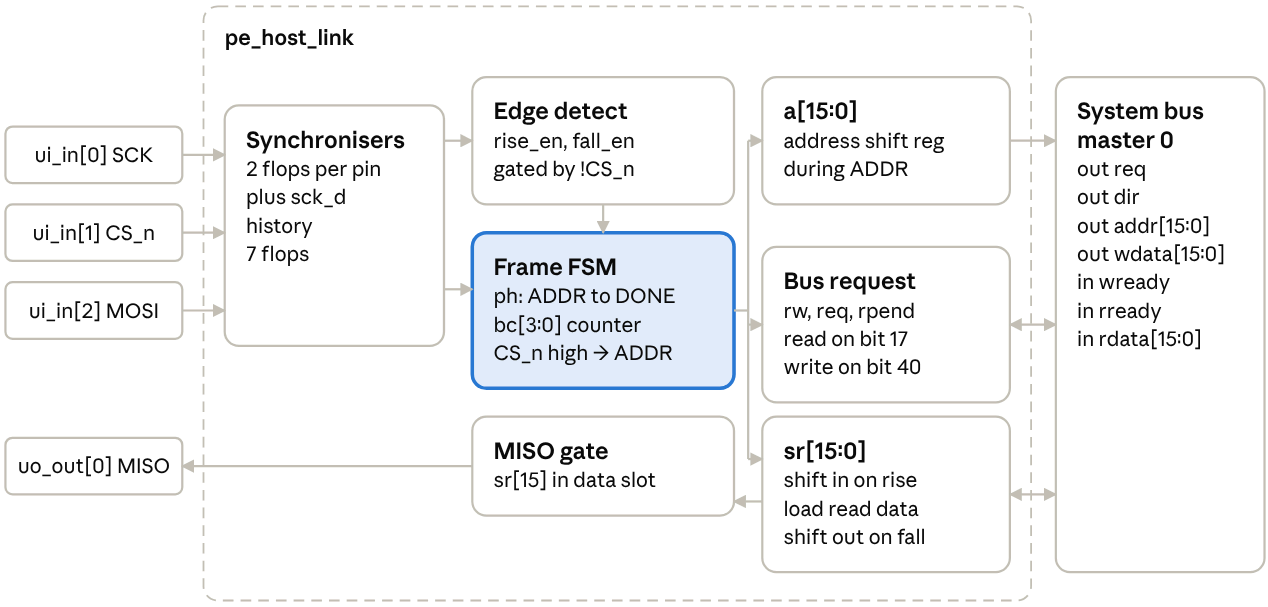
Documentation convention D-DOC-1: draw.io only for the micro-architecture concept diagram, WaveDrom for timing, no hand-drawn gate-level schematics. The equations in §4 are the single definition of the logic; a schematic can be generated from the RTL with Yosys later. Source files: claude/host-link-uarch.drawio and the WaveDrom sources in Appendix A.

2. Micro-architecture

The host link is a synchroniser, a frame state machine and two 16-bit shift registers that drive one bus request (F1).

One SPI frame in, one bus access out, in 48 flops on clk

F1 pe_host_link micro-architecture · SCK is sampled as data, never used as a clock



Frame on MOSI: A[7:0] · A[15:8] · CMD (0x00 write, 0x80 read) · D[7:0] · D[15:8], bytes MSB-first.
MISO returns R[7:0] · R[15:8] in the last two bytes. CS_n high aborts; a write needs all 40 bits.

F1 · pe_host_link micro-architecture

- Synchronisers: SCK, CS_n and MOSI each pass two flops; a third SCK flop (sck_d) keeps the previous value for edge detection.
- Edge detect: one-clock pulses rise_en and fall_en, only while CS_n is low.
- Frame FSM: the phase ADDR, CMD, DATA, DONE and the bit counter bc. It enables every shift and raises the bus request.
- a[15:0]: collects the address during ADDR and holds it until the next frame.
- sr[15:0]: shifts write data in on SCK rises, or loads read data and shifts it out on SCK falls.
- Bus request: rw, req and rpend. A read is requested on frame bit 17, a write after frame bit 40; req holds until the bus accepts.
- MISO gate: drives sr[15] only in the read-data bytes and 0 otherwise.

3. Frame and interface

A frame is 40 SCK in mode 0 (CPOL 0, CPHA 0), bytes MSB-first, 16-bit fields low byte first (figure T1).

Frame bits	MOSI	MISO (read)	Notes
1–8	A[7:0]	0	Address low byte
9–16	A[15:8]	0	Address high byte
17–24	CMD: 0x00 write, 0x80 read	0	Only bit 7 (frame bit 17) is decoded; the read is issued on it
25–32	D[7:0] (write)	R[7:0]	MOSI ignored on a read
33–40	D[15:8] (write)	R[15:8]	The write is issued after bit 40

SCK after bit 40 is ignored until CS_n goes high. CS_n high at any time restarts the frame, and a write with fewer than 40 bits is dropped. MISO is 0 outside the read-data bytes (HL-08).

```
module pe_host_link (
  input  wire        clk,
  input  wire        rst_n,      // synchronised reset (async assert, sync
release, G-02)
  input  wire        sck_i,      // ui_in[0]
  input  wire        csn_i,      // ui_in[1]
  input  wire        mosi_i,     // ui_in[2]
  output wire        miso_o,     // uo_out[0]
  output wire        req_o,      // bus master 0
  output wire        dir_o,      // 1 = write, 0 = read
  output wire [15:0] addr_o,
  output wire [15:0] wdata_o,
  input  wire        wready_i,   // write accepted this clock
  input  wire        rready_i,   // read accepted this clock (side effect at
the edge)
  input  wire [15:0] rdata_i     // valid the clock after rready_i
);
```

Port	Dir	Timing
sck_i, csn_i, mosi_i	in	Asynchronous; first flop only, no logic before it (P3). Host keeps SCK ⩽ clk/10

Port	Dir	Timing
miso_o	out	AND3 of flops; changes 2–3 clocks after a host SCK fall
req_o	out	Flop; set by the frame logic, cleared on the edge ending the accepting clock
dir_o	out	!rw; stable while req_o = 1
addr_o	out	{a[7:0], a[15:8]}, wiring only; stable while req_o = 1
wdata_o	out	{sr[7:0], sr[15:8]}, wiring only; stable while req_o = 1 and dir_o = 1
wready_i, rready_i	in	Combinational from the interconnect in the same clock
rdata_i	in	Captured by rpend exactly one clock after rready_i

The interconnect must give the host priority on every clock (BUS-03), answer an unmapped address with ready = 1 and rdata = 0, and not make wready_i or rready_i depend combinationally on wdata.

4. Storage and logic

All 48 flops are sg13cmos5l dfrbpq with CLK = clk, RESET_B = rst_n and reset value 0. No flop is clock-gated (G-01), so every enable is a MUX2. There are no FIFOs, no write-hold register and no address incrementer.

Flop	Bits	Purpose
sck_s1, sck_s2, sck_d	3	SCK synchroniser and edge history
csn_s1, csn_s2	2	CS_n synchroniser
mosi_s1, mosi_s2	2	MOSI synchroniser
ph[1:0]	2	Frame phase: 00 ADDR, 01 CMD, 10 DATA, 11 DONE
bc[3:0]	4	Bit counter
a[15:0]	16	Address shift register
sr[15:0]	16	Write data in, read data out

Flop	Bits	Purpose
rw	1	R/W bit of the current command
req	1	Bus request
rpend	1	Read data due this clock

Control logic:

```
sel      = !csn_s2
rise_en  = sck_s2 & !sck_d & sel
fall_en  = !sck_s2 & sck_d & sel

ph_addr  = !ph1 & !ph0      ph_cmd  = !ph1 & ph0
ph_data  =  ph1 & !ph0      ph_done =  ph1 & ph0
tc       = bc == 15         (AND4)
b8       = bc == 8          (AND4, three inputs inverted)
bnz      = bc != 0          (OR4)

inc      = bc + 1           (i0 = !bc0, i1 = bc1^bc0, i2 = bc2^(bc1&bc0), i3
= bc3^(bc2&bc1&bc0))
ld8      = ph_addr & tc     (ADDR -> CMD: bc + 1 = 0, so OR-ing 8 into bit 3
loads 8)
bc_d     = sel ? (rise_en ? {i3|ld8, i2, i1, i0} : bc) : 4'd0

ph_inc   = rise_en & tc & !ph_done
ph_d     = sel ? (ph_inc ? ph + 1 : ph) : 2'd0          (DONE saturates)
```

Datapath and bus request:

```
a_sh     = rise_en & ph_addr
a_d      = a_sh ? {a[14:0], mosi_s2} : a

cap      = rise_en & ph_cmd & b8                        (frame bit 17, the
R/W bit)
rw_d     = cap ? mosi_s2 : rw
set_rd   = cap & mosi_s2
set_wr   = rise_en & ph_data & tc & !rw                 (frame bit 40 of a
write)

acc      = req & (rw ? rready_i : wready_i)
req_d    = set_rd | set_wr | (req & !acc)
```

```
rpend_d  = req & rw & rready_i

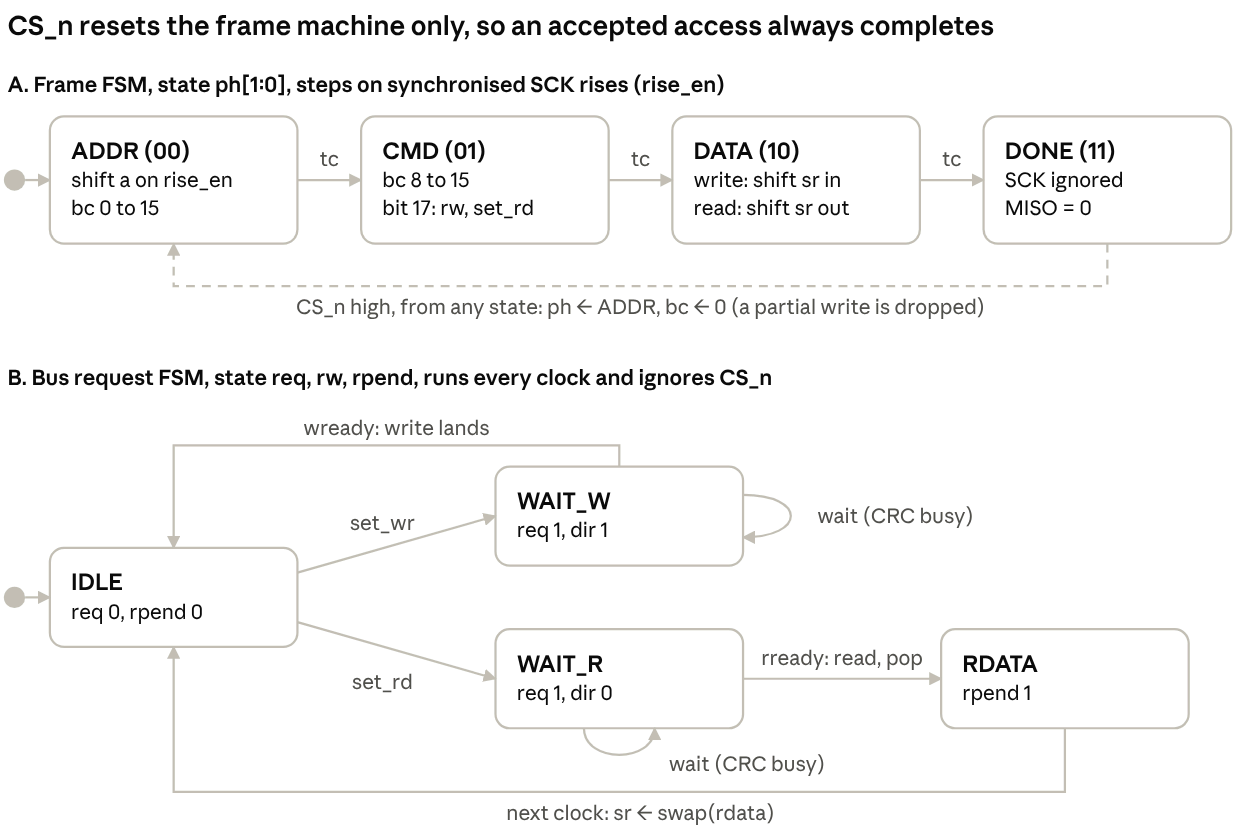
sr_shift = (rise_en & ph_data & !rw) | (fall_en & ph_data & rw & bnz)
sr_d     = rpend    ? {rdata_i[7:0], rdata_i[15:8]} :
           sr_shift ? {sr[14:0], mosi_s2} : sr

miso_o   = sr[15] & rw & ph_data
dir_o    = !rw
addr_o   = {a[7:0], a[15:8]}
wdata_o  = {sr[7:0], sr[15:8]}
```

- During a read, sr fills from mosi_s2 as it shifts. Those bits never reach sr[15] within 15 shifts, so they never appear on MISO; this saves a gate.
- bnz blocks the shift on the first DATA fall, so R[7] stays on MISO until the host samples bit 25 (figure T2).
- CS_n high resets only ph and bc. req, rpend, rw, a and sr keep their values, so an accepted bus access always completes.

5. State machines

Two independent machines run the block: the frame machine follows SCK, and the bus request machine follows the bus (F2).



F2 · frame FSM and bus request FSM

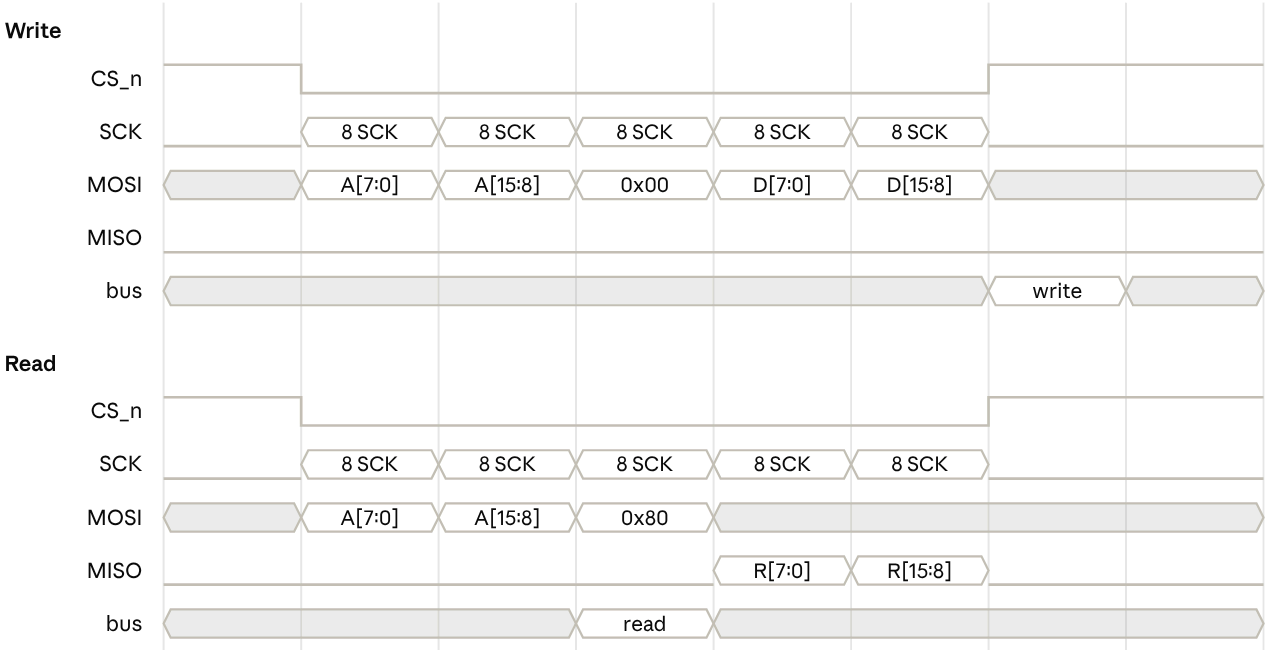
The frame machine's state is ph[1:0]; tc (bc = 15) ends each phase, and the counter loads 8 on entering CMD so that CMD lasts 8 bits. The bus request machine's state is held in req, rw and rpend; CS_n high never clears it, so a read or write the bus has accepted always finishes.

6. Timing

A read needs at most 13 clocks to have data ready, and the frame gives it about 75 at clk/10. Figures T1–T5 are drawn from the WaveDrom sources in Appendix A. E1, E2, … are the clk edges after a pin edge.

One frame is 40 SCK and one 16-bit bus access

T1 Frame format · one cell = one byte (8 SCK), bytes MSB-first



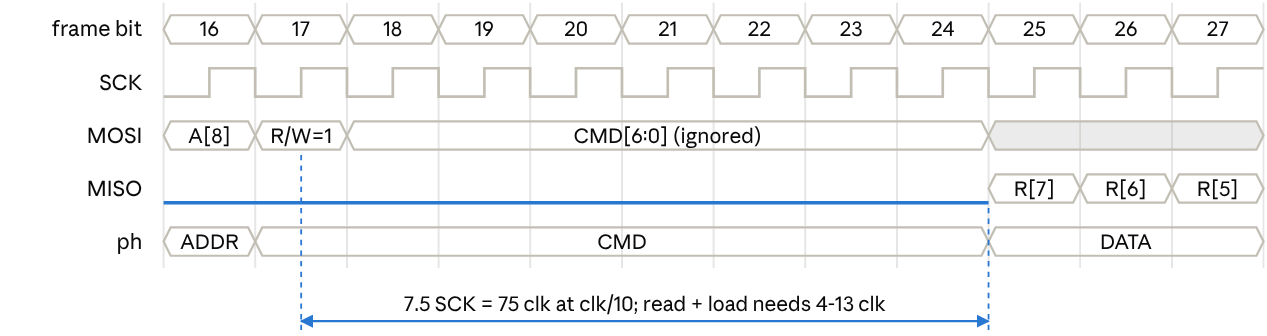
Read is issued on the first CMD bit (frame bit 17); write after frame bit 40

T1 · frame format

The address arrives first, so the bus read can start on the R/W bit, before the data bytes begin.

Read data has 7.5 SCK periods to reach MISO after the R/W bit

T2 Read turnaround · frame bits 16 to 27 · SPI mode 0: change on SCK fall, sample on SCK rise



MISO is 0 until the DATA phase; the first DATA fall does not shift (bc = 0), so R[7] is held for bit 25

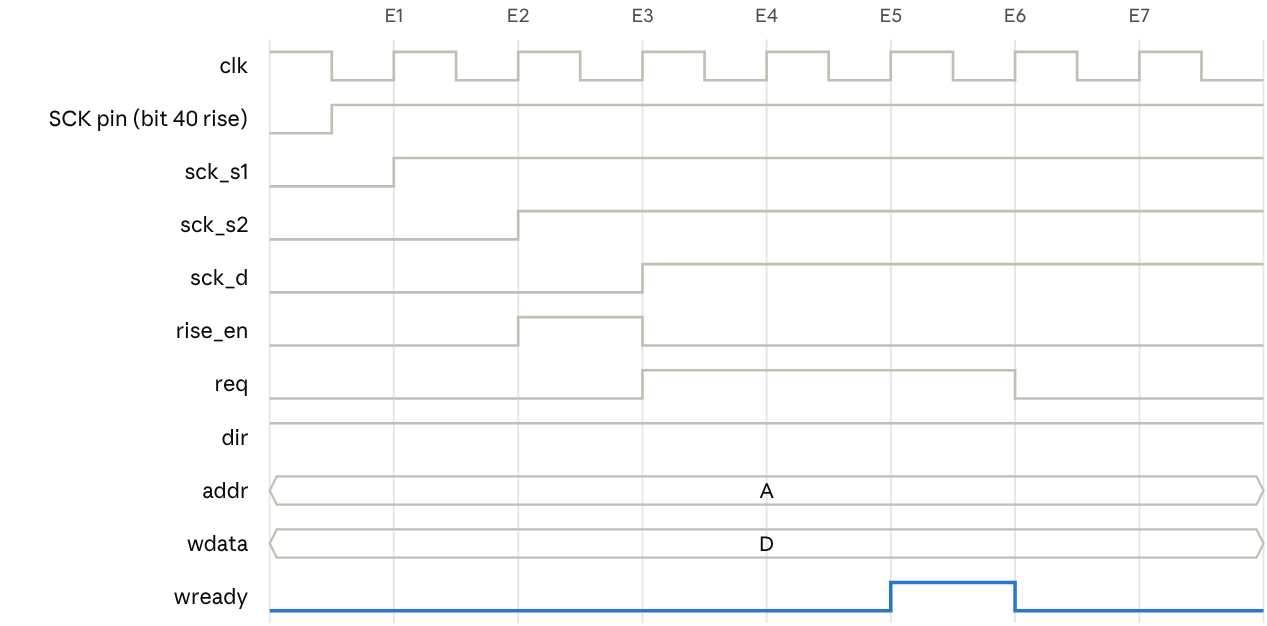
T2 · read turnaround, frame bits 16 to 27

Write, after the 40th SCK rise at the pin (figure T3):

Edge	Event
E1	sck_s1 = 1
E2	sck_s2 = 1, so rise_en = 1 in the next clock
E3	set_wr: req ← 1
E4 + k	Bus write takes effect (k = clocks of wready_i = 0; CRC busy ≤ 8)

A write lands 4 clocks after bit 40, plus one per wait clock

T3 Write handshake · one cell = one clk · example with 2 CRC wait clocks



Write takes effect at the edge ending the wready clock (E6 here, E4 with no wait). req drops at that edge

T3 · write handshake, 2 wait clocks

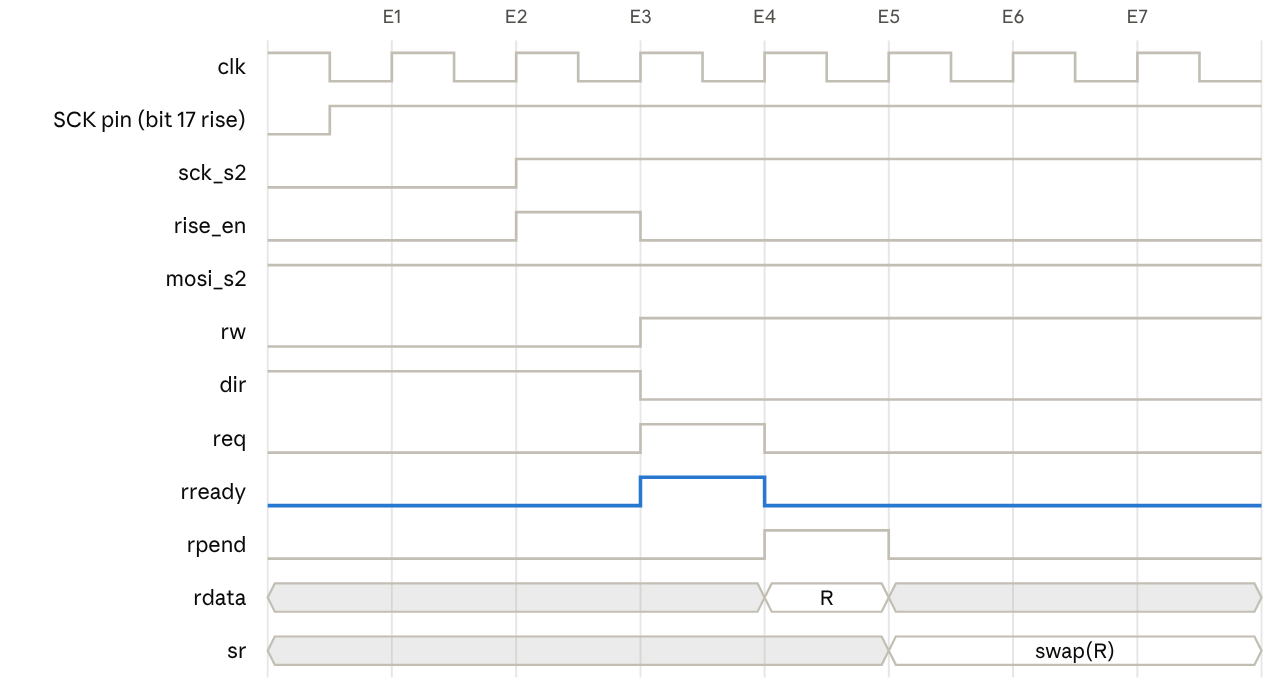
Read, after the 17th SCK rise, the R/W bit (figures T2 and T4):

Edge	Event
E3	cap: rw ← 1, req ← 1
E4 + k	rready_i: access accepted, side effect, req ← 0, rpend ← 1
E5 + k	sr ← swap(rdata_i)

Edge	Event
7.5 SCK after the R/W bit	Host samples the first MISO bit (≈ 75 clocks at clk/10; ⩽ 13 used)

With no wait, read data is in sr 5 clocks after the R/W bit

T4 Read handshake · one cell = one clk · no wait



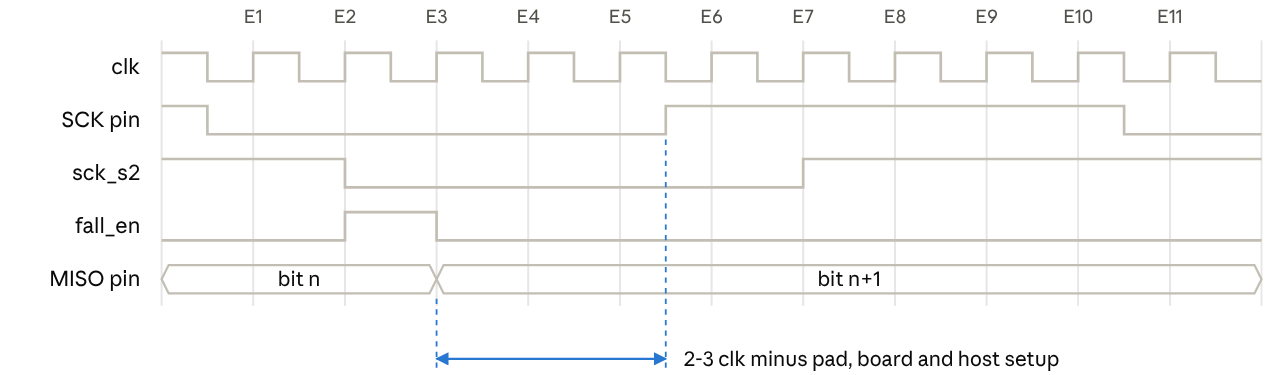
Access and FIFO B pop at E4 (end of the rready clock); rdata valid the next clock; sr loaded at E5. Each wait clock adds one

T4 · read handshake, no wait

MISO per bit (figure T5): sr shifts 2–3 clocks after the host's SCK fall, and the host samples 5 clocks after the fall at clk/10. The margin of 2–3 clocks must cover clock-to-Q, the AND3, the output pad, the board and the host's setup; IHP CMOS5L pad delays are not characterised yet (P6). The model passes at clk/8 with a 1-clock board delay and fails at clk/7.

MISO changes 2 to 3 clocks after the SCK fall, sampled 5 clocks after it

T5 MISO bit timing at SCK = clk/10 · one cell = one clk



sr shifts at E3 after the SCK fall; the host samples on the next SCK rise, 5 clk after the fall

T5 · MISO bit timing at clk/10

7. Clocking, reset, area and timing

The block has one clock and treats SCK as data; it fits in about 3.8k µm² of the 14.0k µm² shared with the bus and system controller (HL-09).

- Clocking: only `clk`. No generated or gated clocks.
- Reset (D-HL-8): all 48 flops reset to 0, the synchronisers included. O-10 allows unreset synchronisers; resetting them gives X-free simulation at no area cost, since it is the same cell. For 2 clocks after reset csn_s2 = 0 (looks selected), but sck_s2 = sck_d = 0 gives no edges, so nothing happens.
- CDC: SCK, CS_n and MOSI each pass two flops of equal depth, so their edge order is kept to within one clock. MOSI is stable for half an SCK period before the rise, so sampling mosi_s2 on rise_en is safe. Mark sck_s1, csn_s1 and mosi_s1 as synchroniser flops in the flow (P3).

Area item	Count	Unit µm²	Area µm²
dfrbpq flops	48	48.9888	2,351
MUX2 (a 16, sr 32, bc 4, ph 2, rw 1, acc 1)	56	18.144	1,016
Other gates (≈ 12 INV, 14 AND2, 7 AND3, 4 AND4, 4 OR, 4 XOR2, 1 NOR2)	≈ 46	5–15	≈ 430
Total before CTS			≈ 3,800

Flop, MUX2, AND2 and XOR2 areas come from the project's area_model.py. INV, AND3, AND4 and OR areas are assumptions, because the area model does not list them. Chip-level CTS and buffering (+10%) are counted separately.

Internal timing is short:

- Longest path inside the block: bc → AND4 (tc) → AND3 (ph_inc) → MUX2 → AND2 → ph D, about 5 gate levels.
- Bus path: wready_i/rready_i → MUX2 → AND2 (acc) → INV → AND2 → OR3 → req D, 5 levels plus the interconnect's decode. Budget it in the bus chapter.
- Every bus output comes straight from a flop (dir through one inverter).

8. Host rules, verification and decisions

The host driver must follow four rules:

1. H-1: Mode 0, SCK ⩽ clk/10, bytes MSB-first, 40 SCK per frame.
2. H-2: At least 1 SCK period from the last SCK rise of a frame to the first SCK rise of the next (model minimum: 9 clocks with an 8-clock CRC wait).
3. H-3: Do not abort a read once its command byte has started; the read, and any FIFO B pop, still happens.
4. H-4: Halt a sub-core before writing its program or configuration; writes while it runs are ignored (G-07).

Verification (VER-07, adapted to A-CHG-2):

Test	Check
V1 Writes	One write per frame to SRAM, each sub-core window, pin layer, system controller and an unmapped address; correct addr and data; exactly one bus write
V2 Reads	Same classes; unmapped reads 0; FIFO B pops exactly once per read frame
V3 Phase sweep	Every SCK phase against clk: clk/12, clk/10 and clk/8 pass; clk/7 recorded as the limit
V4 Waits	wready/rready held low 0–8 clocks on reads and writes

Test	Check
V5 Abort	CS_n high after each bit 1–40: no write below 40 bits; a read only from bit 17 on
V6 Back-to-back	Gap at the H-2 minimum with 8-clock waits
V7 Misc	SCK after bit 40 ignored; CMD bits 6:0 ignored; reset mid-frame; a 1-clock CS_n glitch
V8 MISO	0 during writes, outside the data bytes and when idle
V9 Gate level	V1–V8 on the netlist, no X after reset; T3 and T4 matched clock for clock

The Python cycle model and gate-equation model already pass V3–V6 and the abort cases: clk/12 to clk/8 pass, clk/7 fails, 8-clock waits pass and a 9-clock gap passes. Move both scripts into the repository with the RTL.

Decisions and open items:

ID	Decision	Reason
D-HL-1	Frame format of §3 (O-01)	Address-first gives read turnaround without a dummy word
D-HL-2	Own synchronisers for the host pins (PL-14)	FILT and LOOPBACK never affect the host link
D-HL-4	CRC busy makes the host link wait (O-07)	Same rule as the main core
D-HL-5	Read committed on the R/W bit	A CRC-busy read needs up to 13 clocks; later would fail at clk/10
D-HL-6	No write-hold register (owner)	Area; H-2 keeps addr and wdata stable
D-HL-7	Bus protocol req/dir/wready/rready/rdata (owner)	Matches BUS-02; side effect exactly once
D-HL-8	All flops reset, synchronisers included	X-free simulation at no area cost
D-DOC-1	draw.io for concept, WaveDrom for timing (owner)	Equations stay the one definition of the logic

Still unverified: the RP2350's SPI behaviour (CS_n timing, gaps between frames, SCK rate), IHP CMOS5L pad delays, the INV/AND3/AND4/OR cell areas and the interconnect's timing.

Appendix A. WaveDrom sources

Each block renders in the WaveDrom editor (wavedrom.com/editor.html) or any WaveDrom 3.x plugin. One cell is one byte in T1, one SCK period in T2 and one clk period in T3–T5.

T1 Frame format

```
{
 "signal": [
  ["Write",
    {"name": "CS_n", "wave": "10....1."},
    {"name": "SCK", "wave": "0=====0.", "data": ["8 SCK", "8 SCK", "8 SCK",
"8 SCK", "8 SCK"]},
    {"name": "MOSI", "wave": "x=====x.", "data": ["A[7:0]", "A[15:8]",
"0x00", "D[7:0]", "D[15:8]"]},
    {"name": "MISO", "wave": "0......."},
    {"name": "bus", "wave": "x.....=x", "data": ["write"]}
  ],
  {},
  ["Read",
    {"name": "CS_n", "wave": "10....1."},
    {"name": "SCK", "wave": "0=====0.", "data": ["8 SCK", "8 SCK", "8 SCK",
"8 SCK", "8 SCK"]},
    {"name": "MOSI", "wave": "x===x...", "data": ["A[7:0]", "A[15:8]",
"0x80"]},
    {"name": "MISO", "wave": "0...==0.", "data": ["R[7:0]", "R[15:8]"]},
    {"name": "bus", "wave": "x..=x...", "data": ["read"]}
  ]
 ],
 "head": {"text": "Host link frames: 40 SCK, one 16-bit word per frame (bytes
MSB-first)"},
 "foot": {"text": "Read is issued on the first CMD bit (frame bit 17); write
after frame bit 40"},
 "config": {"hscale": 2}
}
```

T2 Read turnaround

```
{
 "signal": [
  {"name": "frame bit", "wave": "============", "data": ["16", "17", "18",
"19", "20", "21", "22", "23", "24", "25", "26", "27"]},
  {"name": "SCK", "wave": "nnnnnnnnnnnn"},
  {"name": "MOSI", "wave": "===......x..", "data": ["A[8]", "R/W=1",
"CMD[6:0] (ignored)"], "node": ".a"},
  {"name": "MISO", "wave": "0........===", "data": ["R[7]", "R[6]", "R[5]"],
"node": ".........b"},
  {"name": "ph", "wave": "==.......=..", "data": ["ADDR", "CMD", "DATA"]}
 ],
 "edge": ["a~>b 7.5 SCK = 75 clk at clk/10; read + load needs 4-13 clk"],
 "head": {"text": "Read turnaround, SPI mode 0 (MOSI/MISO change on SCK fall,
sampled on SCK rise)"},
 "foot": {"text": "MISO is 0 until the DATA phase; the first DATA fall does
not shift (bc = 0), so R[7] is held for bit 25"},
 "config": {"hscale": 1}
}
```

T3 Write handshake

```
{
 "signal": [
  {"name": "clk", "wave": "p......."},
  {"name": "SCK pin (bit 40 rise)", "wave": "01......", "phase": 0.5},
  {"name": "sck_s1", "wave": "01......"},
  {"name": "sck_s2", "wave": "0.1....."},
  {"name": "sck_d", "wave": "0..1...."},
  {"name": "rise_en", "wave": "0.10...."},
  {"name": "req", "wave": "0..1..0."},
  {"name": "dir", "wave": "1......."},
  {"name": "addr", "wave": "=.......", "data": ["A"]},
  {"name": "wdata", "wave": "=.......", "data": ["D"]},
  {"name": "wready", "wave": "0....10.", "node": "......a"}
 ],
 "head": {"text": "Write handshake (example: CRC busy, 2 wait clocks)",
"tick": 0},
 "foot": {"text": "Write takes effect at the edge ending the wready clock (E6
here, E4 with no wait). req drops at that edge"}
}
```

T4 Read handshake

```
{
 "signal": [
  {"name": "clk", "wave": "p......."},
  {"name": "SCK pin (bit 17 rise)", "wave": "01......", "phase": 0.5},
  {"name": "sck_s2", "wave": "0.1....."},
  {"name": "rise_en", "wave": "0.10...."},
  {"name": "mosi_s2", "wave": "1.......", "node": ""},
  {"name": "rw", "wave": "0..1...."},
  {"name": "dir", "wave": "1..0...."},
  {"name": "req", "wave": "0..10..."},
  {"name": "rready", "wave": "0..10..."},
  {"name": "rpend", "wave": "0...10.."},
  {"name": "rdata", "wave": "x...=x..", "data": ["R"]},
  {"name": "sr", "wave": "x....=..", "data": ["swap(R)"]}
 ],
 "head": {"text": "Read handshake (no wait)", "tick": 0},
 "foot": {"text": "Access and FIFO B pop at E4 (end of the rready clock);
rdata valid the next clock; sr loaded at E5. Each wait clock adds one"}
}
```

T5 MISO bit timing

```
{
 "signal": [
  {"name": "clk", "wave": "p.........."},
  {"name": "SCK pin", "wave": "10....1....0", "phase": 0.5, "node":
"......b"},
  {"name": "sck_s2", "wave": "1.0....1...."},
  {"name": "fall_en", "wave": "0.10........"},
  {"name": "MISO pin", "wave": "=..=........", "data": ["bit n", "bit n+1"],
"node": "...a"}
 ],
 "edge": ["a<->b 2-3 clk minus pad, board and host setup"],
 "head": {"text": "MISO bit timing at SCK = clk/10", "tick": 0},
 "foot": {"text": "sr shifts at E3 after the SCK fall; the host samples on
the next SCK rise, 5 clk after the fall"}
}
```